1

How the Command Line Works

Before we dive into practical Linux commands, you need to have a basic understanding of how the command line works. This chapter will give you that understanding.

For new developers, we'll explore the initial skills that you need to get started on the Linux command line. For those with a little more experience, there are still some nuances to discover, such as the difference between "shell" and "command line." It pays to know the difference!

In this chapter, we will cover the following topics:

- The basic idea of a command-line interface, or CLI
- The form that commands take
- How command arguments work and how they look when you're typing commands and when you're looking up documentation
- An introduction to "the shell," and how it differs from the "command line"
- The core rules that the shell uses to look up commands

To begin with, we'll start off with the basic idea of a command-line interface. We will get ourselves up to speed with how a CLI works and run through a quick example.

In the beginning...was the REPL

What is a **command-line interface** (**CLI**)? It's a text-based environment for interacting with your computer that:

1. Reads some input from you,
2. Evaluates (or processes) that input,
3. Prints some output to the screen in response, and then
4. Loops back to the beginning to repeat that process.

Let's look at what happens at each step, on a practical level, with the `ls` (list) command, which you'll see in a few pages. For now, it's enough to know that the `ls` command lists the contents of a directory.

Step	**What it means**
1. Read input	You type the `ls` command and press *Enter*.
2.Evaluate command	The shell looks up the `ls` binary, finds it, and tells the machine to execute it.
3. Print output	The `ls` command emits some text – the names of all files and directories it found – and the shell prints that output to your terminal window.
4. Loop back to 1 (repeat the process)	Once the programs called by the command have exited, repeat the process by accepting more user input.

If you read steps 1-4 again, you'll notice that the first letter of each step spells "REPL", which is a common way of referring to this kind of Read-Eval-Print Loop in the languages that invented and refined this workflow, such as Lisp.

To put this into programming terms, you can translate the REPL instructions above into code:

```
while (true) { // the loop
  print(eval(read()))
}
```

Indeed, you can create a REPL capable of doing basic calculations with just a few lines of code in most programming languages. Here's a one-liner "shell" program written in Perl:

```
perl -e 'while (<>){print eval, "\n"}'
1+2
3
```

Here, we write the code as a parameter, printing the output of the evaluation as long as there is input to read from. At the end, we append a new line and exit.

This program is tiny, but it's enough to implement an interactive Read-Eval-Print Loop in a command-line environment – a **shell**. The shells you'll use in Linux and Unix are significantly more complex than this Perl mini-shell, but the principles are the same.

The point is simple: as a developer, you might already be using REPLs without realizing it, because almost all modern scripting languages come with one. In essence, the Linux (or macOS, or other Unix) command line functions like the "interactive shells" that interpreted languages give you. So even if you're not familiar with the Lisp REPL, the Perl snippet above should remind you of a very basic Ruby or Python shell.

Now that you understand the basic mechanics of the command-line interfaces you'll be using in Linux, you're ready to try out your first commands. To do that, you'll need to know the correct command-line syntax to use.

Command-line syntax (read)

All REPLs start by reading some input. On the Linux command line, commands that the shell reads in need to have the correct syntax. Commands take this basic form:

```
commandname options
```

In programming terms, you can think of the command name as a function name, and the options as any number of arguments that will be passed to that function. This is important, because there is no single fixed syntax for all the options – each command defines which parameters it will accept. Because of this, the shell can do very little to validate a command's correctness beyond checking that the command maps to an executable.

Note

The terms "program" and "command" are used interchangeably in this chapter. There's a very slight difference because some shell builtins are defined in the shell's code and are therefore not technically separate programs of their own, but you don't need to worry about it – leave that distinction to the Unix greybeards.

Let's dive into more complex variations on this "command [options]" syntax, which you'll see frequently:

```
command [-flags,] [--example=foobar] [even_more_options ...]
```

This is the conventional format you'll see used in help documentation such as the program manual pages (manpages) included in most Linux environments, and it's fairly simple:

- `command` is the program you're running
- Items in brackets are optional, and brackets with ellipses (`[xyz ...]`) tell you that you can pass zero or more arguments here
- `-flags` means any valid option ("flag," in Unix-speak) for that program, e.g. `-debug or -foobar`

Some programs will also accept short and long versions of a parameter, usually denoted by single- vs. double-hyphenation: so `-l` and `--long` might do the same thing. It's not consistent across commands, though; this kind of behavior requires that the command's creator implemented short and long arguments that set the same parameter.

Not all commands will implement all these ways of passing configuration when invoking them, but these represent the most common forms you'll see.

By default, a space denotes the end of an argument, so just like in most programming languages, an argument string that includes spaces must be single- or double-quoted. You'll read more about this in *Chapter 12*, *Automating Tasks with Shell Scripts*.

In just a moment, we'll follow the process of how the shell interprets a command that you issue using this syntax, but first we want to clearly define the difference between two sometimes-interchangeable terms we've been using in this chapter: "command line" and "shell."

Command line vs. shell

In this book, we refer to a "command-line environment." We define this as any text-based environment that acts as a kind of REPL, specifically for interacting with the operating system, programming language interpreter, database, etc. A "command-line" environment or interface describes the general idea of how you're interacting with a system.

But there's a more specific term which we'll use here: shell.

A shell is a specific program that implements this command-line environment and lets you give it text commands. Technically, there are lots of different shells which provide the same kind of REPL-based command-line environment, often for wildly different things:

- Bash is a common shell environment for interacting with Linux and Unix operating systems.
- Popular databases like Postgres, MySQL, and Redis all provide a shell for developers to interact with and run commands in.
- Most interpreted languages provide a shell environment to speed up development. In these, valid commands are simply programming language statements. See `irb` for Ruby, the interactive Python shell, etc.

- Zsh (the Z shell) is an alternative operating system shell (like Bash), which you might see on some developers' laptops if they've customized their environments.

When we talk about a *shell* in this book, we're referring to a Unix shell (generally Bash), which is a command-line interface specifically designed to let you interact with the underlying Linux or Unix operating system.

How does the shell know what to run? (evaluate)

After *reading* in a command, the shell needs to *evaluate* it, by executing a program, fetching some information, or doing something else that's actually useful to you.

Note

Such a detailed description of how shells work may seem tedious at first, but we promise that this knowledge will come in handy when you have to troubleshoot an issue with a missing or incorrectly permissioned program.

When you type a command like `foobar -option1 test.txt` in a shell like Bash and press *Enter*, a few things happen:

1. If the command has a path specified, it will be used. This can take various forms:
    - A full path, like `/usr/bin/foobar` in the command `/usr/bin/foobar -option1 test.txt`.
    - A relative path, like the current working directory in the command `./foobar-option1 test.txt` (the `.` denotes the current directory, which we'll cover in the *Absolute vs. Relative Filepaths* section below; this command essentially says "please execute the "foobar" file that's in my current directory").
    - The path may be based on variables and symbols either in:
        - The shell's environment (env vars) like `$HOME/foobar`, or
        - Provided by the shell, like `~/foobar` (the `~` character means "this user's home directory")
2. If not, the shell checks to see whether it knows what `foobar` means:
    - It could be a built-in shell command.
    - It could be an *alias*, which is a way to set up macros or shortcuts for commands.
3. If not, the shell generally looks at the `$PATH` environment variable, which contains a few different locations to check for commands: `/bin`, `/usr/bin`, `/sbin`, etc. Users can add locations to this `$PATH` list, and various software will modify your `$PATH`: version managers for scripting languages, Python's virtual environments, and many other programs make heavy use of this mechanism. The shell tries those places specified in your `$PATH`, in the order it finds them in the `$PATH` variable, to see if any of them contain an executable with the name `foobar`.

If the shell still hasn't found anything, it'll return an error like `bash: foobar: command not found:`.

On the other hand, if at any point the shell indeed finds an executable file named `foobar`, it executes that file and passes `-option1` and `test.txt` (in that order) as arguments.

At this point, the shell knows what program to use to evaluate the command, and it does so. As the command is evaluated, any output is printed to the user, completing the third step of the REPL process. Now all that's left to do is to loop back to the beginning and start the process over again, accepting another command as input from the user.

The shell tries its best to guess which program the user wants to run, using the general process we outlined above to resolve ambiguity. However, ambiguity can be a bad thing and lead to misunderstandings or bugs. During troubleshooting, you'll often want to find out which command is really being run. To accomplish this, you can use the command `which <command>`, which will print the full path (or the alias or script being run) and will let you know whether that command is a shell builtin. Depending on the system, `which` might not be available. In these situations, you can use `command –v` instead. This is the POSIX equivalent, which we'll learn about next:

```
bash-3.2$ which ls
/bin/ls
bash-3.2$ command -v ls
/bin/ls
```

A quick definition of POSIX

Wikipedia tells us that "the **Portable Operating System Interface** (**POSIX**) is a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems." Practically speaking, it's an attempt at defining some common standards between Unix systems, which can otherwise have wildly different sets of basic commands available.

POSIX basically says things like, "every POSIX-compatible OS should have a list command called `ls`"; in this case, "every POSIX-compatible OS should have a way to check to see if a matching executable exists for a given command name."

If your scripts need to be portable across Unix operating systems, restricting yourself to POSIX commands is a good thing to do. However, it's still not a guarantee – many extremely popular Linux distributions divert from POSIX in numerous ways, most of which you won't notice until they bite you.